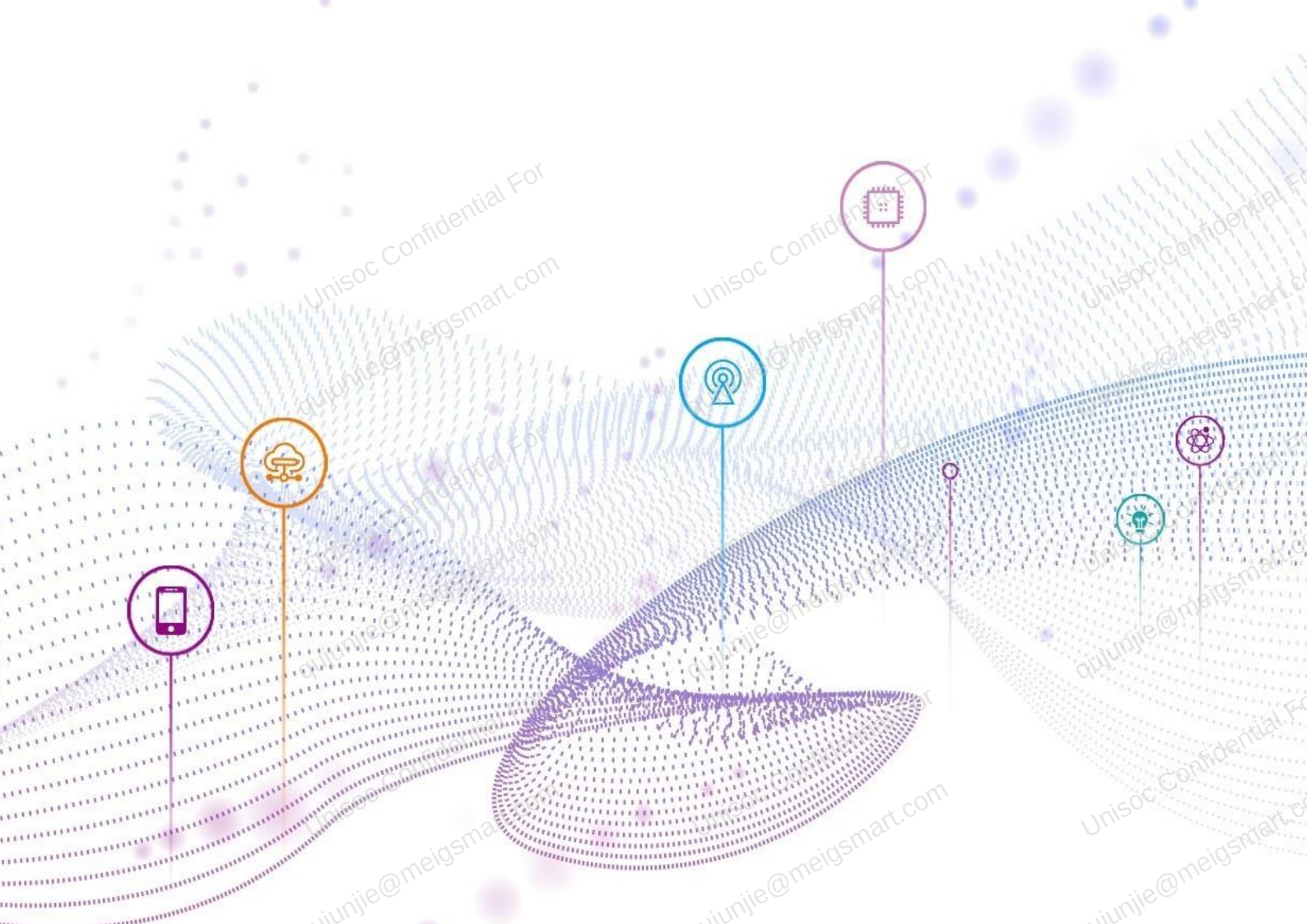




Android 14~16 Sensor Hub 客制化指导手册

文档版本

V1.4

发布日期

2025-06-05

版权所有 © 紫光展锐（上海）科技股份有限公司。保留一切权利。

本文件所含数据和信息都属于紫光展锐（上海）科技股份有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负责任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

商标声明

紫光展锐、UNISOC、、展讯、Spreadtrum、SPRD、锐迪科、RDA 及其他紫光展锐的商标均为紫光展锐（上海）科技股份有限公司及/或其子公司、关联公司所有。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

免责声明

本文档可能包含第三方内容，包括但不限于第三方信息、软件、组件、数据等。紫光展锐不控制且不对第三方内容承担任何责任，包括但不限于准确性、兼容性、可靠性、可用性、合法性、适当性、性能、不侵权、更新状态等，除非本文档另有明确说明。在本文档中提及或引用任何第三方内容不代表紫光展锐对第三方内容的认可、承诺或保证。

用户有义务结合自身情况，检查上述第三方内容的可用性。若需要第三方许可，应通过合法途径获取第三方许可，除非本文档另有明确说明。

紫光展锐（上海）科技股份有限公司



前言

概述

本文档介绍了 sensor hub 客制化过程，内容涵盖 sensor hub 架构、sensor hub 工程使用说明、编写 sensor 驱动、sensor hub 客制化指导、debug 方法以及常见问题及处理方法。

读者对象

本文档主要适用于需要对 sensor hub 代码进行调试、开发及验证的人员，或者有兴趣了解 sensor hub 的人员。

符号约定

在本文中可能出现下列符号，每种符号的说明如下。

符号	说明
 说明	用于突出重要或关键信息、补充信息和小窍门等。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害。
 注意	用于突出容易出错的操作。 “注意”不是安全警示信息，不涉及人身、设备及环境伤害。
 警告	用于可能无法恢复的失误操作。 “警告”不是危险警示信息，不涉及人身及环境伤害。

变更信息

文档版本	发布日期	修改说明
V1.4	2025-06-05	更新 3.8 中断实现
V1.3	2025-05-20	- 更新 3.2.2 i2c_dev_info 成员说明 - 补充 I2C 和 I3C 常用接口说明 - 增加 3.11 sensor 运行在 DDR 上 - 更新 4.4 移除 sensor - 更新 5.2 常用系统文件节点调试 sensor hub 命令：增加 SPI 调试节点说明

文档版本	发布日期	修改说明
		文档名由《Android 14~15 Sensor Hub 客制化指导手册》修改为《Android 14~16 Sensor Hub 客制化指导手册》
V1.2	2025-04-28	- 增加 2.7 内存预留宏控 - 增加 3.2.3 i3c_dev_info - 增加 3.4.1.4 sensor_i2c_read - 增加 3.4.1.5 sensor_i2c_write - 增加 3.4.2 I3C 常用接口 3.5 多供兼容实现增加 name 匹配多种情况 - 将原有的“添加 AP 与 sensor hub 交互命令”拆分为针对 Kernel 5.15 和 Kernel 6.6 的两个小节。
V1.1	2024-07-16	- 增加 3.6.2 闭源库 overlay 说明。 3.2.2 i2c_dev_info 增加对 interface_num 的说明。
V1.0	2023-08-02	第一次正式发布。

关键字

sensor、传感器、sensor hub、CH

目 录

1 sensor hub 架构	1
2 sensor hub 工程使用说明	3
2.1 编译 sensor hub 工程	3
2.2 下载 sensor hub 产物	3
2.3 配置 sensor hub 功能宏	4
2.4 新增 sensor hub board	5
2.5 链接 sensor hub 库	6
2.6 划分 sensor hub 内存段	7
2.7 内存预留宏控	7
3 编写 sensor hub sensor 驱动	8
3.1 ERR_MSG 枚举	8
3.2 常用结构体	10
3.2.1 sensor_driver	10
3.2.2 i2c_dev_info	12
3.2.3 i3c_dev_info	13
3.2.4 sensor_info_t	14
3.3 驱动初始化注册宏	15
3.4 常用接口	15
3.4.1 I2C 常用接口	15
3.4.2 I3C 常用接口	19
3.4.3 tx_thread_sleep	22
3.4.4 kal_thread_sleep	23
3.4.5 SENSORHUB_TRACE	23
3.4.6 sensor_process_base_sensor_data	24
3.4.7 sensor_process_onchange_sensor_data	24
3.4.8 driver_common_sensor_direction_correction	25
3.4.9 时间获取接口	25
3.5 多供兼容实现	26
3.6 overlay 实现	26
3.6.1 实现步骤	27
3.6.2 闭源库 overlay 说明	29
3.7 sensor 校准实现	31
3.8 中断实现	31
3.9 距感公共逻辑	32
3.9.1 距感结构体	32
3.9.2 距感公共函数	34

3.10 光感/距感 raw_data 获取.....	35
3.11 sensor 运行在 DDR 上.....	35
4 sensor hub 客制化指导	37
4.1 sensor bring up	37
4.1.1 新增 sensor hub 驱动.....	37
4.1.2 配置 AP 端 sensor 类型宏.....	38
4.2 Kernel 5.15 版本添加 AP 与 sensor hub 交互命令	38
4.2.1 SIPC 消息格式.....	39
4.2.2 AP 发送 sensor hub 接收.....	39
4.2.3 sensor hub 发送 AP 接收.....	41
4.3 Kernel 6.6 版本添加 AP 与 sensor hub 交互命令	42
4.3.1 SIPC 消息格式.....	42
4.3.2 AP 发送 sensor hub 接收.....	43
4.3.3 sensor hub 发送 AP 接收.....	44
4.4 移除 sensor.....	46
5 Debug 方法	47
5.1 sensor hub log 获取.....	47
5.1.1 adb 获取实时 log.....	47
5.1.2 获取历史 log.....	47
5.2 常用系统文件节点调试 sensor hub 命令	47
6 常见问题及处理方法.....	49
6.1 sensor hub 工程编译提示内存段溢出错误	49
6.1.1 问题现象.....	49
6.1.2 问题处理.....	49
7 参考文档.....	50

图目录

图 1-1 sensor hub 整体框图	1
图 2-1 sensorhub 工程配置窗口	4
图 2-2 contexthub 工程配置窗口	5
图 3-1 SRAM memory overlay 结构	27

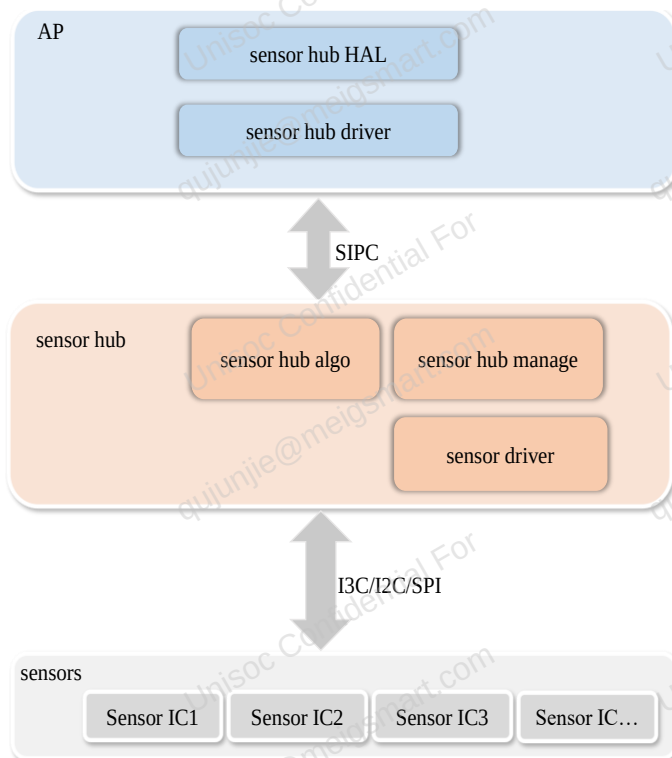
表目录

表 1-1 AP 和 sensor hub 核中运行的代码	1
表 5-1 常用的系统文件节点调试命令	47

1 sensor hub 架构

图 1-1 是 sensor hub 的整体框架图，外部 sensor IC 通过 I3C、I2C、SPI 挂载于 sensor hub 核，sensor hub 核通过 SIPC 通信方式与 AP 核进行交互。

图1-1 sensor hub 整体框图



AP 和 sensor hub 核中运行的各部分代码如表 1-1 所示。

表1-1 AP 和 sensor hub 核中运行的代码

模块名	模块功能	代码路径
sensor hub HAL	实现 Android 定义的标准 sensor HAL 接口。 Android sensor HAL 介绍文档链接： https://source.android.com/devices/sensors/sensors-multihal	• vendor/sprd/modules/sensors/sprd_subhal • vendor/sprd/modules/sensors/libsensorhub

模块名	模块功能	代码路径
sensor hub driver	主要负责： - 将 HAL 层下发的命令传递给 sensor hub。 - 将 sensor hub 反馈的信息及上报的 sensor 事件传递给 HAL 层。 - 提供调试接口。	/drivers/iio/sprd_hub
sensor hub manage	- 处理 AP 发送的命令。 - 采集和上报 sensor 数据。	- sp sys: bsp/sensorhub/public/sensor_hub_sprd - ch sys: bsp/contexthub/modules/sensorhub/sensor_hub_sprd
sensor hub algo	sensor 算法源码，以库方式释放。	代码闭源
sensor driver	sensor 的驱动代码，被 sensor hub manage 调用。	- sp sys: bsp/sensorhub/public/sensor_hub_sprd/public/system/sensor_driver - ch sys: bsp/contexthub/modules/sensorhub/sensor_hub_sprd/public/system/sensor_driver

说明

- 本文档中出现的 sp sys 包括的平台有：SC9863A、SC9863A1、SC9863T、SC9863TW、UMS512(T)、UMS312、UMS9230(E/H/M/S/T/W)、UIS7861、UIS7863、UIS7865；ch sys 包括的平台有：UMS9620(T/S/W)、UIS6870W、UIS7870SC、UIS7885、UMS9621(S)、UMS9632(S)、UMS9360，后文不再赘述。
- Android sensor 介绍文档链接：<https://source.android.com/devices/sensors>

2 sensor hub 工程使用说明

sensor hub 工程使用 GCC 编译器进行编译，使用 CMake+Kconfig 对代码编译进行配置。与 kernel 代码的 Makefile + Kconfig 不同在于，sensor hub 使用的是 CMake，增删编译链时需要修改对应的 CMakeLists 或.cmake 文件。

说明

CMake 是 Makefile 的上游工具，CMake 执行后将生成 Makefile 文件。

2.1 编译 sensor hub 工程

sensor hub 工程可以在代码全编时进行编译，也可以单独编译。

sensor hub 工程单独编译步骤：

- 步骤 1 进入 bsp 目录。
- 步骤 2 执行命令 `source build/envsetup.sh`。
- 步骤 3 执行命令 `lunch`，选择工程。
- 步骤 4 编译工程。
 - 若是 sp sys，执行命令 `make sensorhub`。
 - 若是 ch sys，执行命令 `make contexthub`。

说明

- sp sys 工程编译生成的可执行产物 (.bin) 在 `bsp/sensorhub/public/build/(项目名)` 目录下。
- ch sys 工程编译生成的可执行产物 (.bin) 在 `bsp/contexthub/out/(项目名)/(board 名)` 目录下。

----结束

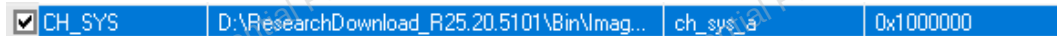
2.2 下载 sensor hub 产物

完整的 pac 包中包含 sensorhub bin 或 contexthub bin。为调试方便，常需单独下载 bin。

单独下载 bin 的方法：

- 步骤 1 使用 ResearchDownload 工具加载一个完整的 pac 包。
- 步骤 2 在 Download settings 中勾选下载分区，并取消对其他分区的勾选。
 - 若是 sp sys，则勾选以下分区：

☒ DFS	D:\ResearchDownload_R25.20.5101\Bin\mag...	pm_sys_a	0x100000
---	--	----------	----------
 - 若是 ch sys，则勾选以下分区：



步骤 3 用待下载 bin 的路径覆盖上图中的路径。

步骤 4 按正常下载 pac 包的方式进行下载。

说明

- sensorhub bin 默认已签名，如无特殊需求，不需要再进行签名。
- contexthub bin 不需要签名。

----结束

2.3 配置 sensor hub 功能宏

工程功能宏文件 defconfig 路径：

- sp sys: bsp/sensorhub/public/project/(项目名)
- ch sys: bsp/contexthub/device/CH/(项目名)/(board 名)/

直接修改 defconfig 文件可以达到配置宏的目的，但当宏之间存在依赖时，这种做法可能会导致编译报错，因此建议使用 make sensorhub_menuconfig 或 make contexthub_menuconfig 命令进行功能宏配置。

功能宏配置步骤：

步骤 1 进入 bsp 目录。

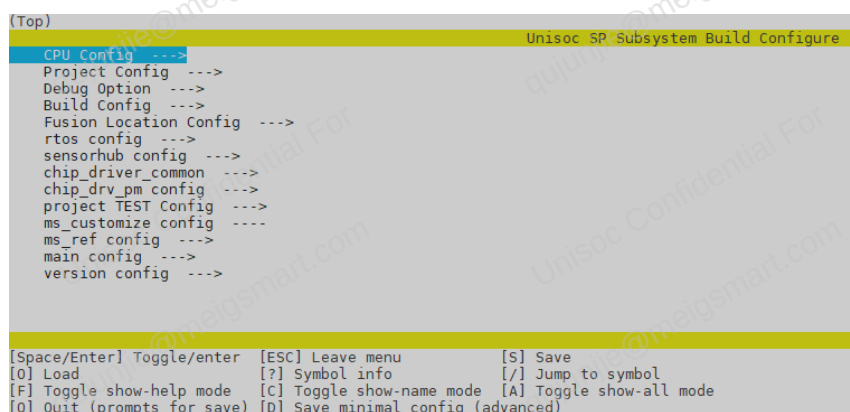
步骤 2 执行命令 source build/envsetup.sh。

步骤 3 执行命令 lunch，选择工程。

步骤 4 配置 sensor hub 功能宏。

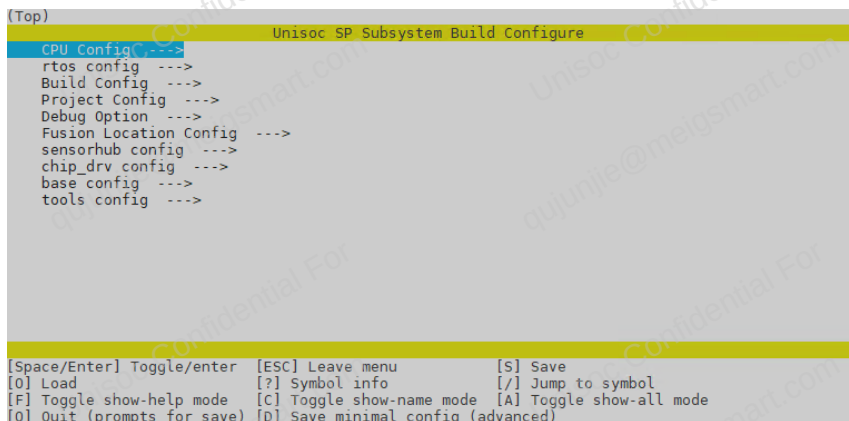
- sp sys: 执行命令 make sensorhub_menuconfig，将弹出如图 2-1 所示窗口。

图2-1 sensorhub 工程配置窗口



- ch sys: 执行命令 make contexthub_menuconfig，将弹出如图 2-2 所示窗口。

图2-2 contexthub 工程配置窗口



步骤 5 按窗口下方提示进行操作。

窗口下方部分符号说明如下：

- /: 搜索并跳转至对应的功能宏项
- S: 保存修改
- Q: 退出本次配置
- Enter: 开启或关闭功能宏

----结束

2.4 新增 sensor hub board

当已有的 sensorhub 工程或 contexthub 工程配置不能兼容新项目要求时，可按如下步骤新增 board:

步骤 1 将 bsp board 与新增的 board 关联。

- sp sys:
 - a. 在 bsp/device/(项目名)/androidx/common/目录中新增 sensorhub.cfg 文件。
该路径中的 androidx 代表不同的 Android Vendor 版本，比如 androidt，androidv。其他地方类似，后文不再赘述。
 - b. 以 UMS512 项目新增 ums512_1h10 sensor hub board 为例，在 sensorhub.cfg 文件中配置如下内容：

```
export BSP_SENSORHUB_PROJECT="sharkl5pro_cm4"
export BSP_SENSORHUB_BOARD="ums512_1h10"
```
- ch sys:
 - a. 在 bsp/device/(项目名)/androidx/common/目录中新增 contexthub.cfg 文件。
 - b. 以 UMS9620 项目新增 ums9620_2h10 contexthub board 为例，在 contexthub.cfg 文件中配置如下内容：

```
export BSP_CONTEXTHUB_PROJECT="QogirN6Pro_CH"
export BSP_CONTEXTHUB_BOARD="ums9620_2h10"
```

说明

工程编译时会逐级往下在以下目录中查找 sensorhub.cfg 或 contexthub.cfg 文件，后查找到的 sensorhub.cfg 或 contexthub.cfg 文件会覆盖先查找到的 sensorhub.cfg 或 contexthub.cfg 文件配置。

- bsp/device/(项目名)/androidx/common
- bsp/device/(项目名)/androidx/(board 名)/(board 名)_base
- bsp/device/(项目名)/androidx/(board 名)/(board 名)_xxx(本次 lunch 选中的工程)

步骤 2 增加 defconfig 文件。

- sp sys:

在 bsp/sensorhub/public/project/(项目名)/(board 名)/目录中新增 defconfig 文件。

下面以 SC9863A 项目新增 s9863a14c10 为例进行说明。在已有的 board 中，如 bsp/sensorhub/public/project/sharkl3_cm4/s9863a1h10，复制一份已有的 defconfig 文件，重命名为 s9863a14c10_defconfig，然后粘贴到 bsp/sensorhub/public/project/sharkl3_cm4/s9863a14c10 目录。

- ch sys:

在 bsp/contexthub/device/CH/(项目名)/(board 名)/目录中新增 defconfig 文件。

下面以 UMS9620 项目新增 ums9620_2h10 为例进行说明。在已有的 board 中，如 bsp/contexthub/device/CH/QogirN6Pro_CH/ums9620_1h10，复制一份已有的 defconfig 文件，重命名为 ums9620_2h10_defconfig，然后粘贴到 bsp/contexthub/device/CH/QogirN6Pro_CH/ums9620_2h10 目录。

步骤 3 配置 defconfig 文件。

按 [2.3 配置 sensor hub 功能宏](#) 所述配置功能宏：

- 执行 lunch，选择工程。
- 按项目需求进行配置功能。

步骤 4 编译新增 board。

按 [2.1 编译 sensor hub 工程](#) 所述编译工程。

- 执行 lunch，选择工程。
- 编译成功后，提交 sensorhub.cfg 或 contexthub.cfg，以及 lunch 选中工程的对应 defconfig 文件。

----结束

2.5 链接 sensor hub 库

当 sensor 驱动使用了第三方库或者需要新增以库形式释放的 sensor 算法时，可按如下步骤链接库：

步骤 1 将库放到使用了第三方库的驱动目录下。

步骤 2 在对应 sensor 类型下的.cmake 文件中使用以下命令链接该库。

```
cp_append_export_library(${PROJECT_SOURCE_ROOT}/sensor_hub_sprd/public/system/sensor_driver/xxx_drivers/(库文件夹)/(库名字.a)
```


说明

命令中的 xxx 代表 sensor 类型。

----结束

2.6 划分 sensor hub 内存段

sensor hub 工程在 lds 文件中划分内存段，lds 文件中使用到的宏定义在 mem_cfg.h 中。

- sp sys:
 - lds 文件：在 bsp/sensorhub/public/ms_customize 目录中
 - mem_cfg.h 文件：在 bsp/sensorhub/public/ms_customize/source/product/config/(项目名)目录中
- ch sys: lds 文件和 mem_cfg.h 文件在 bsp/contexthub/device/CH/(项目名)/(board 名)目录中

sensor hub 工程中 sensor 相关的代码保存和运行在 RAM 上。RAM 通常被划分为 CODE、HEAP 和 RW 三段。通常不需要修改内存段配置，但当编译碰到内存段溢出错误时，参见 [6 常见问题及处理方法](#) 进行处理。

2.7 内存预留宏控

UMS9360 平台预留了内存，以便在有需求时释放更多的内存来进行定制化的功能开发。内存预留宏控为 mem_cfg.h 中的 RESERVE_CODE_SIZE。目前仅 UMS9360 平台有此宏控。

文件路径：bsp/contexthub/device/CH/QogirL8_CH/ums9360_1h10/mem_cfg.h

```
37: /* reserve existing IRAM RW for cus */  
38: #define RESERVE_CODE_SIZE 20000
```

若想使用这块预留内存，将 20000 设置为 0 即可获取到约 156KB 大小的 IRAM_RW 空间释放。此宏控数值每减少 1，将释放 8 Byte 的 IRAM_RW 内存。请根据实际情况使用此宏控。使用此宏控后，如果编译碰到内存段溢出错误，参见 [6 常见问题及处理方法](#) 进行处理。

3 编写 sensor hub sensor 驱动

每个型号的 sensor 都需要有对应的驱动，驱动代码路径如下：

- sp sys: bsp/sensorhub/public/sensor_hub_sprd/public/system/sensor_driver
- ch sys: bsp/contexthub/modules/sensorhub/sensor_hub_sprd/public/system/sensor_driver

如果在上述路径找不到使用的 sensor 型号所对应的驱动，则需要编写对应的 sensor 驱动。

本章将介绍编写驱动会用到的信息，便于开发者编写 sensor 驱动。

3.1 ERR_MSG 枚举

【功能】

定义错误消息。

【定义】

```
typedef enum{  
    I3C_FAIL=-19,  
    SEND_DATA_TO_AP_FAILED=-18,  
    ERR_I2cComandBlock=-17,  
    ERR_I2cSlave=-16,  
    ERR_I2cMaster=-15,  
    ERR_UsageFault=-14,  
    ERR_BusFault=-13,  
    ERR_MemManage=-12,  
    ERR_FLASH=-11,  
    ERR_TASK_BLOCK=-10,  
    SEND_QUEUE_FULL=-9,  
    DRIVER_CHECK_CHIP_ID_FAIL=-8,  
    DRIVER_ENABLE_FAIL=-7,  
    DRIVER_DISABLE_FAIL=-6,  
    DRIVER_GETDATA_FAIL=-5,  
    I2C_FAIL=-4,  
    DRIVER_NO_USE=-3,  
}
```

```
SENSORS_NO_INITIAL=-2,
FAIL=-1,
NO_ERROR=0,
NO_DATA=1,
} ERR_MSG;
```

【成员说明】

成员	说明
I3C_FAIL	I3C 失败
SEND_DATA_TO_AP_FAILED	向 AP 发送数据失败
ERR_I2cComandBlock	I2C 命令阻塞
ERR_I2cSlave	I2C 从设备错误
ERR_I2cMaster	I2C 主设备错误
ERR_UsageFault	使用错误
ERR_BusFault	总线故障
ERR_MemManage	内存管理错误
ERR_FLASH	Flash 错误
ERR_TASK_BLOCK	任务阻塞
SEND_QUEUE_FULL	队列已满
DRIVER_CHECK_CHIP_ID_FAIL	驱动检查 chip ID 失败
DRIVER_ENABLE_FAIL	打开驱动失败
DRIVER_DISABLE_FAIL	关闭驱动失败
DRIVER_GETDATA_FAIL	驱动获取数据失败
I2C_FAIL	I2C 失败
DRIVER_NO_USE	驱动未使用
SENSORS_NO_INITIAL	sensor 未初始化
FAIL	其他错误
NO_ERROR	无错误
NO_DATA	无数据

3.2 常用结构体

3.2.1 sensor_driver

【功能】

struct sensor_driver 是 sensor 驱动的抽象，是 sensor 驱动的关键结构体。结构体中的变量用于保存驱动的配置和运行状态，结构体中罗列了驱动提供给 sensor hub manage 调用的所有方法。

【定义】

```
struct sensor_driver {  
    sensor_info_t *sensor_info;  
    struct sensor_data sensor_data;  
    int report_mode;  
    int sensor_driver_handle;  
    int enabled;  
    int sampling_period_us;  
    int sampling_timer;  
    int max_report_latency_us;  
    int sensor_support_status;  
  
    struct sensor_driver* (*init) ();  
    int (*check_id) ();  
    int (*hw_init) ();  
    int (*set_cali_data) (void *cali_data);  
    int (*set_rate) (int sampling_period_us);  
    int (*activate) ();  
    int (*deactivate) ();  
    int (*set_status) ();  
    int (*get_status) ();  
    int (*get_data) (struct sensor_data *sensor_data, uint64_t timestamp);  
    int (*set_mode) (int mode);  
    int (*get_fifo_data) (struct sensor_data *sensor_data);  
    int (*self_test) ();  
    int (*flush) (int sensor);  
    int (*cali_cmd) (int cal_cmd, int cali_type, int golden_sample);  
    int (*get_cali_data) ();  
};
```

【成员说明】

成员	说明
sensor_info	sensor_info 结构体指针，用于获取 sensor_info 内容。
sensor_data	被用做 get_data 函数的入参。
report_mode	数据采集模式。现有轮询和中断两种模式，按需配置。 0：轮询（默认） 1：中断
sensor_driver_handle	驱动 handle 值，用于获取该驱动的 handle 值。
enabled	sensor 的开关状态，默认值为 0，表示关闭状态。
sampling_period_us	轮询模式时 sensor 的采样间隔，默认值 320000μs。
sampling_timer	距上次采样的时间，默认值 40000μs。
max_report_latency_us	设置的最大上报延迟值，默认值 0。
sensor_support_status	sensor 初始化状态，默认值 0。
sensor_driver* (*init) ()	驱动初始化函数指针，在该函数中进行初始化操作。 初始化错误时返回 0，正确时要返回 sensor_driver 结构体的地址，以使 sensor hub manage 能找到该驱动。
(*check_id) ()	检查 ID 函数指针，在该函数中进行检查 ID 的操作。sensor hub manage 暂未调用，常在 init 函数中调用，用以检查是否挂载该 IC。
(*hw_init) ()	IC 初始化函数指针，在该函数中进行 IC 初始化操作。sensor hub manage 暂未调用，常在 init 函数中调用。
(*set_cali_data) (void *cali_data)	设置校准数据函数指针，在该函数中解析并保存 AP 下发的校准结果。
(*set_rate) (int sampling_period_us)	设置采样速率的函数指针，在该函数中进行 IC 速率设置的操作。
(*activate) ()	使能函数指针，在该函数中进行 IC 使能操作。
(*deactivate) ()	关闭函数指针，在该函数中进行 IC 关闭操作。
(*set_status) ()	设置状态函数指针，在该函数中进行 IC 状态设置。sensor hub manage 暂未调用。
(*get_status) ()	获取 IC 数据状态函数指针，在该函数中判断 IC 数据是否准备好。常在 get_data 函数中调用。
(*get_data) (struct sensor_data *sensor_data, uint64_t timestamp)	采集数据函数指针，在该函数中进行 IC 数据采集、数据补偿、数据上报操作。

成员	说明
(*set_mode) (int mode)	设置模式函数指针，在该函数中进行 IC 模式设置。sensor hub manage 暂未调用。
(*get_fifo_data) (struct sensor_data *sensor_data)	获取 FIFO 数据函数指针，在该函数中获取 IC FIFO 内数据。sensor hub manage 暂未调用。
(*self_test) ()	自测试函数指针，在该函数中进行 IC 自测试。sensor hub manage 暂未调用。
(*flush) (int sensor)	清空数据函数指针，在该函数中进行 FIFO 清空操作。sensor hub manage 暂未调用。
(*cali_cmd) (int cal_cmd, int cali_type, int golden_sample)	校准命令解析函数指针，在该函数中接收并保存 AP 校准命令，供校准函数使用。
(*get_cali_data) ()	获取校准结果函数指针，在该函数中向 AP 发送校准状态或校准结果。

3.2.2 i2c_dev_info

【功能】

该结构体用于设置 I2C 相关参数，在 I2C 读写函数中作为函数入参。

【定义】

```
struct i2c_dev_info {
    uint16_t interface_freq;
    uint32_t interface_num;
    uint8_t slave_addr;
    uint32_t reg_addr_len;
}
```

【成员说明】

成员	说明
interface_freq	I2C 时钟频率，默认 400kHz。

成员	说明
interface_num	I2C 总线编号： - UMS9632、UMS9360 平台 0x10: i2c0（默认值） 0x11: i2c1 - 其他平台 0x00: i2c0（默认值） 0x01: i2c1 注意 interface_num（总线编号）是逻辑编号，与原理图中 I2C 总线编号有所差异，具体需查看 pinmap 的映射关系。以 UMS9620（T/S/W）为例，interface_num 为 i2c0 时，对应原理图中的 i2c2，而 interface_num 为 i2c1 时，对应原理图中的 i2c5。
slave_addr	从设备地址。
reg_addr_len	从设备寄存器地址位数，默认 1。如 0x68 为 1 位，0x6868 为 2 位。

3.2.3 i3c_dev_info

【功能】

该结构体用于设置 I3C 相关参数，在 I3C 读写函数中作为函数入参。

📖说明

sp sys 平台不支持 I3C，该结构体仅适用于 ch sys 平台。

【定义】

```
struct i3c_dev_info {
    uint16_t interface_freq;
    uint32_t interface_num;
    uint8_t slave_addr;
    uint32_t reg_addr_len;
    uint8_t dynamic_address;
}
```

【成员说明】

成员	说明
interface_freq	I3C 时钟频率，默认 1MHz。

成员	说明
interface_num	I3C 总线编号： 0x00: i3c0（默认值） 0x01: i3c1 注意 interface_num（总线编号）是逻辑编号，与原理图中 I2C 总线编号有所差异，具体需查看 pinmap 的映射关系。以 UMS9360 为例，interface_num 为 0x00 时，即 i3c0，对应原理图中的 i2c2。
slave_addr	从设备地址。
reg_addr_len	从设备寄存器地址位数，默认为 1 位。如 0x68 为 1 位，0x6868 为 2 位。
dynamic_address	动态地址。

3.2.4 sensor_info_t

【功能】

该结构体用于填充 sensor 的信息，sensor 驱动代码中可使用。该结构体内容将被发送至 AP，部分信息在 sensor hub driver 的 sensor_info 节点中显示。

【定义】

```
typedef struct {
    char name[20];
    char vendor[20];
    int32_t version;
    int32_t sensor_type;
    float maxrange;
    float resolution;
    float power;
    int32_t mindelay_us;
    uint32_t fifo_reserved_event_count;
    uint32_t fifo_max_event_count;
    int32_t maxdelay_us;
} sensor_info_t
```

【成员说明】

成员	说明
name[20]	IC 名字，建议带上类型信息，在 sensor_info 中显示，如：acc_bmi270。
vendor[20]	IC 供应商信息，在 sensor_info 中显示。
version	IC 版本信息，在 sensor_info 中显示。
sensor_type	sensor 类型，取于 sensor_type 枚举。
maxrange	sensor 支持测量数据的最大范围。
resolution	sensor 原始数据分辨率，常用于 get_data 函数中对原始数据进行处理。
power	sensor 功耗值。
mindelay_us	sensor 支持的最快采样时间。
fifo_reserved_event_count	为该 sensor 保留的 FIFO 数量。
fifo_max_event_count	FIFO 中能保存该 sensor 事件的最大值。
maxdelay_us	sensor 支持的最慢采样时间。

3.3 驱动初始化注册宏

sensor 驱动中需要使用 DRIVER_INIT 宏来注册初始化函数。当设备开机时，sensor hub 启动后，sensor hub manage 会依次调用使用 DRIVER_INIT 宏注册的 sensor 驱动初始化函数。各驱动初始化函数被调用的顺序与驱动编译顺序相关，先被编译到的先调用。

说明

sensor 驱动初始化函数执行成功时要返回该驱动 sensor_driver 结构体的指针，失败则返回 0。

3.4 常用接口

本章对驱动代码中常用到的平台接口进行说明。

3.4.1 I2C 常用接口

I2C 常用接口声明在以下文件中：

- ch sys:
 - Android 13 vnd:
 - bsp/contexthub/modules/sensorhub/sensor_hub_sprd/public/system/ixc_driver/sensors_i2c_operate.h

- Android 15 vnd:
bsp/contexthub/modules/sensorhub/sensor_hub_sprd/public/system/ixc_driver/i2c_driver/sensors_i2c_operate.h

- sp_sys: bsp/sensorhub/public/sensor_hub_sprd/public/system/i2c_driver/sensors_i2c_operate.h

3.4.1.1 sensors_i2c_reg_read

【功能】

I2C 读函数。

【定义】

```
int sensors_i2c_reg_read (struct i2c_dev_info * sensor_i2c_info, uint8_t *reg_addr, uint8_t *buffer, uint32_t byte, uint8_t mask)
```

【参数说明】

参数	说明
sensor_i2c_info	传入定义的 i2c_dev_info 结构体的指针。
reg_addr	传入从设备寄存器地址变量的指针。
buffer	传入 buffer 指针，用于保存读取到的数据。
byte	读取的字节数。
mask	位操作掩码。一次仅能操作 1 字节数据，当 mask=0xff 表示对所有位操作。

【返回值】

- 成功：返回读取到的字节数。
- 失败：返回一个表示失败原因的负数。具体原因，参见 [3.1 ERR_MSG 枚举](#)。

3.4.1.2 sensors_i2c_reg_write

【功能】

I2C 写函数。

【定义】

```
int sensors_i2c_reg_write (struct i2c_dev_info * sensor_i2c_info, uint8_t *reg_addr, uint8_t *buffer, uint32_t byte, uint8_t mask)
```

【参数说明】

参数	说明
sensor_i2c_info	传入定义的 i2c_dev_info 结构体的指针。

参数	说明
reg_addr	传入从设备寄存器地址变量的指针。
buffer	buffer 内保存的值，将被写入从设备寄存器。
byte	写入的字节数。
mask	位操作掩码，一次仅能操作 1 字节数据，当 mask=0xff 表示对所有位操作。

【返回值】

- 成功：返回写入的字节数。
- 失败：返回一个表示失败原因的负数。具体原因，参见 [3.1 ERR_MSG 枚举](#)。

3.4.1.3 sensor_i2c_check_reg_data

【功能】

检查 reg_addr 读出值是否等于 reg_data & mask。

【定义】

```
int sensor_i2c_check_reg_data (struct i2c_dev_info * i2c_info, uint8_t *reg_addr, uint8_t *reg_data, uint8_t mask)
```

【参数说明】

参数	说明
i2c_info	传入定义的 i2c_dev_info 结构体的指针。
reg_addr	传入从设备寄存器地址变量的指针。
reg_data	传入该值，与 reg_addr 地址读出的值进行比较。
mask	位操作掩码，当 mask=0xff 表示对所有位操作。

【返回值】

- 等于：返回 0，表示 NO_ERROR。
- 不等于：返回-1，表示读出值不等于比较值。

3.4.1.4 sensor_i2c_read

【功能】

I2C 读函数。

说明

- ch sys 平台，I2C 和 I3C 下的 I2C mode 会使用该接口。根据 i2c_info 结构体中的 interface_num 的值区分是 I2C，还是 I3C 下的 I2C mode。
- sp sys 平台不支持 I3C，仅 I2C 会使用该接口。

【定义】

```
int sensor_i2c_read (struct i2c_dev_info * i2c_info, uint8_t *reg_addr, uint8_t *buffer, uint32_t bytes)
```

【参数说明】

参数	说明
i2c_info	传入定义的 i2c_dev_info 结构体的指针。
reg_addr	传入从设备寄存器地址变量的指针。
buffer	传入 buffer 指针，用于保存读取到的数据。
bytes	读取的字节数。

【返回值】

- 成功：返回读取到的字节数。
- 失败：返回一个表示失败原因的负数。具体原因，参见 [3.1 ERR_MSG 枚举](#)。

3.4.1.5 sensor_i2c_write

【功能】

I2C 写函数。

说明

- ch sys 平台，I2C 和 I3C 下的 I2C mode 会使用该接口。根据 i2c_info 结构体中的 interface_num 的值区分是 I2C，还是 I3C 下的 I2C mode。
- sp sys 平台不支持 I3C，仅 I2C 会使用该接口。

【定义】

```
int sensor_i2c_write (struct i2c_dev_info * i2c_info, uint8_t *reg_addr, uint8_t *buffer, uint32_t bytes)
```

【参数说明】

参数	说明
i2c_info	传入定义的 i2c_dev_info 结构体的指针。
reg_addr	传入从设备寄存器地址变量的指针。

参数	说明
buffer	传入 buffer 指针，用于保存待写入的数据。
bytes	写入的字节数。

【返回值】

- 成功：返回写入的字节数。
- 失败：返回一个表示失败原因的负数。具体原因，参见 [3.1 ERR_MSG 枚举](#)。

3.4.2 I3C 常用接口

sp sys 平台不支持 I3C，以下接口说明仅适用于 ch sys 平台。

I3C 常用接口声明在以下文件中：

bsp/contexthub/modules/sensorhub/sensor_hub_sprd/public/system/ixc_driver/i3c_driver/i3c_mode/sensors_i3c_operate.h

3.4.2.1 sensor_i3c_dasa

【功能】

I3C 动态地址分配函数。

注意

使用 I3C 接口进行读写及 reset sensor IC 之后必须要进行动态地址分配。

【定义】

```
int sensor_i3c_dasa (struct i3c_dev_info * i3c_info)
```

【参数说明】

参数	说明
i3c_info	传入定义的 i3c_dev_info 结构体的指针。

【返回值】

- 成功：返回 0。
- 失败：返回一个表示失败原因的负数。

3.4.2.2 sensors_i3c_reg_read

【功能】

I3C 读函数。

【定义】

```
int sensors_i3c_reg_read (struct i3c_dev_info *sensor_i3c_info, uint8_t *reg_addr, uint8_t *buffer, uint8_t mask)
```

【参数说明】

参数	说明
sensor_i3c_info	传入定义的 i3c_dev_info 结构体的指针。
reg_addr	传入从设备寄存器地址变量的指针。
buffer	传入 buffer 指针，用于保存读取到的数据。
mask	位操作掩码。一次仅能操作 1 字节数据，当 mask=0xff 表示对所有位操作。

【返回值】

- 成功：返回读取到的字节数。
- 失败：返回一个表示失败原因的负数。具体原因，参见 [3.1 ERR_MSG 枚举](#)。

3.4.2.3 sensors_i3c_reg_write

【功能】

I3C 写函数。

【定义】

```
int sensors_i3c_reg_write (struct i3c_dev_info * sensor_i3c_info, uint8_t *reg_addr, uint8_t *buffer, uint8_t mask)
```

【参数说明】

参数	说明
sensor_i3c_info	传入定义的 i3c_dev_info 结构体的指针。
reg_addr	传入从设备寄存器地址变量的指针。
buffer	buffer 内保存的值，将被写入从设备寄存器。
mask	位操作掩码，一次仅能操作 1 字节数据，当 mask=0xff 表示对所有位操作。

【返回值】

- 成功：返回写入的字节数。
- 失败：返回一个表示失败原因的负数。具体原因，参见 [3.1 ERR_MSG 枚举](#)。

3.4.2.4 sensor_i3c_check_reg_data

【功能】

检查 reg_addr 读出值是否等于 reg_data & mask。

【定义】

```
int sensor_i3c_check_reg_data (struct i3c_dev_info * i3c_info, uint8_t *reg_addr, uint8_t *reg_data, uint8_t mask)
```

【参数说明】

参数	说明
i3c_info	传入定义的 i3c_dev_info 结构体的指针。
reg_addr	传入从设备寄存器地址变量的指针。
reg_data	传入该值，与 reg_addr 地址读出的值进行比较。
mask	位操作掩码，当 mask=0xff 表示对所有位操作。

【返回值】

- 等于：返回 0，表示 NO_ERROR。
- 不等于：返回-1，表示读出值不等于比较值。

3.4.2.5 sensor_i3c_read

【功能】

I3C 读函数。仅 I3C 下 I3C mode 使用。

【定义】

```
int sensor_i3c_read (struct i3c_dev_info * i3c_info, uint8_t *reg_addr, uint8_t *reg_value, uint32_t num)
```

【参数说明】

参数	说明
i3c_info	传入定义的 i3c_dev_info 结构体的指针。
reg_addr	传入从设备寄存器地址变量的指针。

参数	说明
reg_value	传入 reg_value 指针，用于保存读取到的数据。
num	读取的字节数。

【返回值】

- 成功：返回读取到的字节数。
- 失败：返回一个表示失败原因的负数。具体原因，参见 [3.1 ERR_MSG 枚举](#)。

3.4.2.6 sensor_i3c_write

【功能】

I3C 写函数。仅 I3C 下 I3C mode 使用。

【定义】

```
int sensor_i3c_write (struct i3c_dev_info * i3c_info, uint8_t *reg_addr, uint8_t *reg_value, uint32_t num)
```

【参数说明】

参数	说明
i3c_info	传入定义的 i3c_dev_info 结构体的指针。
reg_addr	传入从设备寄存器地址变量的指针。
reg_value	传入 reg_value 指针，用于保存待写入的数据。
num	写入的字节数。

【返回值】

- 成功：返回写入的字节数。
- 失败：返回一个表示失败原因的负数。具体原因，参见 [3.1 ERR_MSG 枚举](#)。

3.4.3 tx_thread_sleep

【功能】

设置当前线程挂起的时间。

📖说明

在 bsp/sensorhub/public/sensor_hub_sprd/ 目录下使用此接口设置当前线程挂起的时间。这个接口只适用于 sp sys 平台。

【定义】`tx_thread_sleep(ms)`**【参数说明】**

参数	说明
ms	输入睡眠多久时间，单位 ms。

【返回值】

返回完成状态。

3.4.4 kal_thread_sleep

【功能】

设置当前线程挂起的时间。

说明

在 `bsp/contexthub/modules/sensorhub/sensor_hub_sprd/` 目录下使用此接口设置当前线程挂起的时间。这个接口只适用于 ch sys 平台。

【定义】`kal_thread_sleep(ms)`**【参数说明】**

参数	说明
ms	输入睡眠多久时间，单位 ms。

【返回值】

无返回值。

3.4.5 SENSORHUB_TRACE

【功能】

打印带 SENSORHUB 前缀的 log。

【定义】`SENSORHUB_TRACE(fmt, args...)`

【返回值】

固定返回 0，无实际意义。

3.4.6 sensor_process_base_sensor_data

【功能】

将连续上报模式 sensor 的 sensor_event 上报给 AP。

【定义】

```
int sensor_process_base_sensor_data(struct sensor_event *sensor_event_data)
```

【参数说明】

参数	说明
sensor_event_data	sensor 事件结构体

【返回值】

固定返回 0，表示 NO_ERROR。

3.4.7 sensor_process_onchange_sensor_data

【功能】

根据 change_status 值上报变化模式 sensor 的 sensor event 到 AP。

【定义】

```
int sensor_process_onchange_sensor_data(struct sensor_event *sensor_event_data, uint8_t change_status)
```

【参数说明】

参数	说明
sensor_event_data	sensor 事件结构体
change_status	是否上报 sensor event 到 AP: • 1: 上报 • 0: 不上报

【返回值】

固定返回 0，表示 NO_ERROR。

注意

上报的 sensor_event 结构体中 sensor_handle 及 timestamp 变量要记得赋值。

3.4.8 driver_common_sensor_direction_correction

【功能】

根据传入的 position 值调整 X、Y、Z 三轴的方向。

【定义】

```
int driver_common_sensor_direction_correction(uint16_t data[3], uint8_t position)
```

【参数说明】

参数	说明
data[3]	X、Y、Z 轴原始数据。
position	position 值对应的方向调整： • 0 表示 X:Y:Z • 1 表示 Y:-X:Z • 2 表示 -X:-Y:Z • 3 表示 -Y:X:Z • 4 表示 Y:X:-Z • 5 表示 X:-Y:-Z • 6 表示 -Y:-X:-Z • 7 表示 -X:Y:-Z

【返回值】

- 成功：返回 0，表示 NO_ERROR。
- 失败：返回 -1，表示 FAIL。

3.4.9 时间获取接口

【功能】

获取实时时间，包括三个函数，每个函数返回的时间精度不同。该时间与 AP 同步过，为 AP boot time。

【定义】

```
uint32_t get_ap_boottime_ms()
```

```
uint64_t get_ap_boottime_us()
```

```
uint64_t get_ap_boottime_ns()
```

【返回值】

按时间精度返回具体的时间。

说明

如需了解 RTOS 系统相关接口，可查看书籍《嵌入式实时操作系统的多线程计算——基于 ThreadX® 和 ARM®》。

3.5 多供兼容实现

同一类型 sensor 有多供，多个硬件工程或板子（board）共用一个软件 board 配置时，可按如下步骤实现多供兼容：

步骤 1 将每一供的驱动都加入编译。

步骤 2 将每个 sensor 驱动初始化函数中的 name 与 sensor_config.h 文件中的 name 匹配。

- 如果匹配到 name，则检查 ID。如果 ID 不匹配，则返回 0。
- 当所有 name 都不匹配时，使用驱动中的默认配置来检查 ID。如果 ID 不匹配，则返回 0。

说明

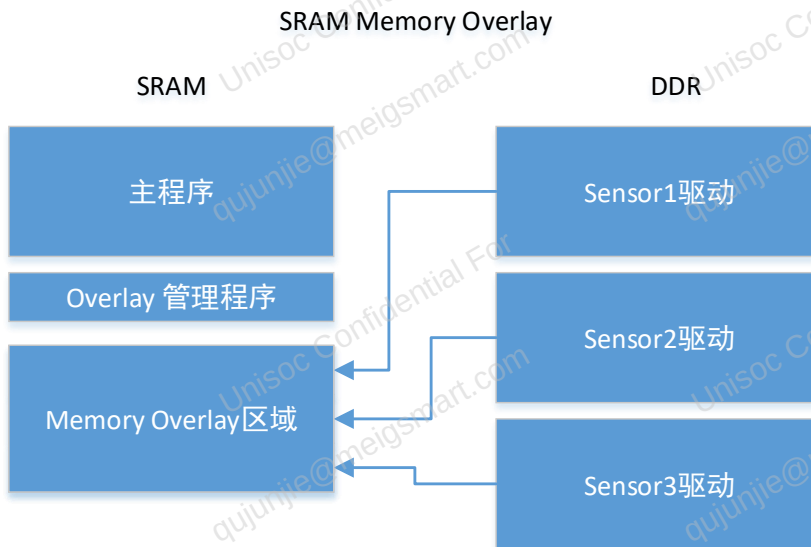
sensor hub 启动后将逐一调用注册的初始化函数。若初始化成功，则返回 sensor_driver 结构体地址并保存。后续 sensor hub manage 将通过该地址，找到 sensor 驱动提供的所有方法。

----结束

3.6 overlay 实现

实际项目开发时，对于同一个项目，同样类型的 sensor 有多个供应商，但 sensor hub 有限的 SRAM 空间难以兼容太多 sensor 驱动。为解决这个问题，引入 memory overlay 方案，其结构如图 3-1 所示。

图3-1 SRAM memory overlay 结构



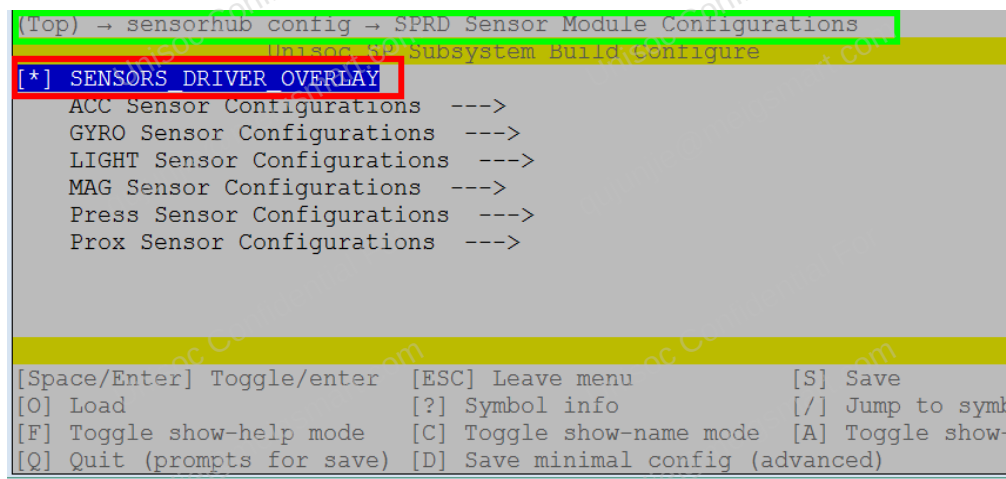
3.6.1 实现步骤

按如下步骤实现 memory overlay:

步骤 1 配置 defconfig 文件。

按 2.3 配置 sensor hub 功能宏所述，配置功能宏 CONFIG_SENSORS_DRIVER_OVERLAY。

- 依次选择 sensorhub config > SPRD Sensor Module Configurations。
- 勾选 SENSORS_DRIVER_OVERLAY，根据页面提示保存并退出。



步骤 2 添加 sensor overlay 管理及 sensor 初始化注册程序。

以加速计 lsm6dsl 和 icm20600 为例，在 acc_lsm6dsl.c 和 acc_icm20600.c 添加 sensor overlay 管理及 sensor 初始化注册程序。

```
#include "sensor_driver_accelerometer_lsm6dsl.h"
#ifdef CONFIG_ACC_DRIVER_OVERLAY
#include "acc_overlay.h"

/* acc lsm6dsl overlay init registration */
overlay_acc_driver_init(lsm6dsl, acc_lsm6dsl_sensor_init);

#else
/* acc lsm6dsl init registration */
DRIVER_INIT(acc_lsm6dsl_sensor_init);
#endif // CONFIG ACC DRIVER OVERLAY
```

步骤 3 在链接脚本（lds 文件）中将每一供的驱动都加入编译。

- sp sys: 在 bsp/sensorhub/public/ms_customize/ 目录中新增多供。
- ch sys: 在 bsp/contexthub/device/CH/(项目名)/(board 名)/ 目录中新增多供。

以 qogirl6_cm4.lds 为例，修改相应目录下的编译脚本，分别对 text 段、data 段、bss 段 overlay 声明加速计 lsm6dsl 和 icm20600。

- text 段

```
/* accelerometer driver text section */
__overlay_acc_text_start__ = SP_RW_IRAM_START;

OVERLAY SP_RW_IRAM_START:
{
    .acc_lsm6dsl_text
    {
        sensor_driver_accelerometer_lsm6dsl.* (.text .text* .rodata .rodata*)
    }
    .acc_icm20600_text
    {
        sensor_driver_accelerometer_icm20600.* (.text .text* .rodata .rodata*)
    }
} > IRAM_RW AT>DDR_CODE
```

- data 段

```
/* accelerometer driver data section */
__overlay_acc_data_start__ = .;

OVERLAY :
{
    .acc_lsm6dsl_data
    {
        sensor_driver_accelerometer_lsm6dsl.* (.data .data*)
    }
    .acc_icm20600_data
    {
        sensor_driver_accelerometer_icm20600.* (.data .data*)
    }
} > IRAM_RW AT>DDR_CODE
. = ALIGN(4);
```

- bss 段

```
/* accelerometer driver bss section */
__overlay_acc_bss_start__ = .;

OVERLAY :
{
    .acc_lsm6dsl_bss
    {
        sensor_driver_accelerometer_lsm6dsl.* (.bss .bss*)
    }
    .acc_icm20600_bss
    {
        sensor_driver_accelerometer_icm20600.* (.bss .bss*)
    }
} > IRAM_RW AT>DDR_CODE
. = ALIGN(4);
```

说明

sensor hub 启动后将逐一调用注册的初始化函数，初始化函数执行时会将驱动拷贝到 IRAM 上执行。若初始化成功，则返回 sensor_driver 结构体地址并保存，且执行同类型其他 sensor 的初始化函数时直接返回 NULL，不执行拷贝内存的动作。后续 sensor hub manage 将通过被保存的地址，找到 sensor 驱动提供的所有方法。

----结束

3.6.2 闭源库 overlay 说明

若涉及到闭源库，在链接闭源库的基础上（参见 2.5 链接 sensor hub 库），不仅需要包含上述驱动代码文件，还需要包含相关的闭源库支持代码。

以 qogirl6_cm4.lds 为例，修改相应目录下的编译脚本，分别对 text 段、data 段、bss 段 overlay 声明地磁 akm99018 代码及使用的静态库 libAK09911.a 中包含的代码。

• text 段

```
/* magnetic driver text section */
__overlay_mag_text_start__ = .;

OVERLAY :
{
    .mag_akm09918_text
    {
        sensor_driver_magnetic_akm09918.* (.text .text* .rodata .rodata*)

        dynamic_api_akm.* (.text .text* .rodata .rodata*)
        akm_cal.* (.text .text* .rodata .rodata*)
        AKMD_APIS.* (.text .text* .rodata .rodata*)
        DOE_APIS.* (.text .text* .rodata .rodata*)
        Measure.* (.text .text* .rodata .rodata*)
        ParameterIO.* (.text .text* .rodata .rodata*)
        SensorsData.* (.text .text* .rodata .rodata*)
        timer.* (.text .text* .rodata .rodata*)

        AKArithOp.* (.text .text* .rodata .rodata* .constdata)
        AKCertification.* (.text .text* .rodata .rodata* .constdata)
        AKDecomp.* (.text .text* .rodata .rodata* .constdata)
        AKDirection6D.* (.text .text* .rodata .rodata* .constdata)
        AKDOECORE.* (.text .text* .rodata .rodata* .constdata)
        AKHDOE.* (.text .text* .rodata .rodata* .constdata)
        AKHFlucCheck.* (.text .text* .rodata .rodata* .constdata)
        AKMathLib.* (.text .text* .rodata .rodata* .constdata)
        AKMDevice.* (.text .text* .rodata .rodata* .constdata)
        AKVersion.* (.text .text* .rodata .rodata* .constdata)
    }
} > IRAM_RW AT>DDR_CODE
. = ALIGN(4);
```

• data 段

```

/* magnetic driver data section */
__overlay_mag_data_start__ = .;

OVERLAY :
{
    .mag_akm09918_data
    {
        sensor_driver_magnetic_akm09918.*    (.data .data*)

        dynamic_api_akm.*                    (.data .data*)
        akm_cal.*                             (.data .data*)
        AKMD_APIS.*                           (.data .data*)
        DOE_APIS.*                             (.data .data*)
        Measure.*                             (.data .data*)
        ParameterIO.*                         (.data .data*)
        SensorsData.*                         (.data .data*)
        timer.*                               (.data .data*)

        AKArithOp.*                           (.data .data*)
        AKCertification.*                     (.data .data*)
        AKDecomp.*                             (.data .data*)
        AKDirection6D.*                       (.data .data*)
        AKDOECore.*                           (.data .data*)
        AKHDOE.*                              (.data .data*)
        AKHFlucCheck.*                        (.data .data*)
        AKMathLib.*                           (.data .data*)
        AKMDevice.*                           (.data .data*)
        AKVersion.*                           (.data .data*)
    }
} > IRAM_RW AT>DDR_CODE
. = ALIGN(4);

```

- bss 段

```

/* magnetic driver bss section */
__overlay_mag_bss_start__ = .;

OVERLAY :
{
    .mag_akm09918_bss
    {
        sensor_driver_magnetic_akm09918.*    (.bss .bss*)

        dynamic_api_akm.*                     (.bss .bss*)
        akm_cal.*                             (.bss .bss*)
        AKMD_APIS.*                           (.bss .bss*)
        DOE_APIS.*                             (.bss .bss*)
        Measure.*                             (.bss .bss*)
        ParameterIO.*                         (.bss .bss*)
        SensorsData.*                         (.bss .bss*)
        timer.*                               (.bss .bss*)

        AKArithOp.*                           (.bss .bss*)
        AKCertification.*                     (.bss .bss*)
        AKDecomp.*                             (.bss .bss*)
        AKDirection6D.*                       (.bss .bss*)
        AKDOECore.*                           (.bss .bss*)
        AKHDOE.*                              (.bss .bss*)
        AKHFlucCheck.*                        (.bss .bss*)
        AKMathLib.*                           (.bss .bss*)
        AKMDevice.*                           (.bss .bss*)
        AKVersion.*                           (.bss .bss*)
    }
} > IRAM_RW AT>DDR_CODE
. = ALIGN(4);

```

其中静态库包含的文件可以在非 overlay 场景下的编译生成物.map 文件中获取。生成物路径参见 [2.1 编译 sensor hub 工程](#) 中的说明。


```
sensor_hub_sprd/public/system/sensor_driver/mag_drivers/akm_cali/libAK09911.a(AKArithOp.o)
sensor_hub_sprd/public/system/sensor_driver/mag_drivers/akm_cali/libAK09911.a(AKCertification.o)
sensor_hub_sprd/public/system/sensor_driver/mag_drivers/akm_cali/libAK09911.a(AKDecomp.o)
sensor_hub_sprd/public/system/sensor_driver/mag_drivers/akm_cali/libAK09911.a(AKDirection6D.o)
sensor_hub_sprd/public/system/sensor_driver/mag_drivers/akm_cali/libAK09911.a(AKDOECore.o)
sensor_hub_sprd/public/system/sensor_driver/mag_drivers/akm_cali/libAK09911.a(AKHDOE.o)
sensor_hub_sprd/public/system/sensor_driver/mag_drivers/akm_cali/libAK09911.a(AKHFlucCheck.o)
sensor_hub_sprd/public/system/sensor_driver/mag_drivers/akm_cali/libAK09911.a(AKMathLib.o)
sensor_hub_sprd/public/system/sensor_driver/mag_drivers/akm_cali/libAK09911.a(AKMDevice.o)
sensor_hub_sprd/public/system/sensor_driver/mag_drivers/akm_cali/libAK09911.a(AKVersion.o)
```

3.7 sensor 校准实现

sensor hub 的校准函数没有另起线程执行，通常在 get_data 函数中调用。当 sensor hub 收到 AP 发送的校准命令后，在 get_data 函数中调用校准函数，用传入的 sensor 原始值进行 sensor 校准结果计算。

对于已支持的 sensor 类型，平台对校准逻辑进行了抽象，将公共校准逻辑放在 sensor_driver_(类型)_common.c 文件中，可直接调用公共逻辑实现校准。如需自定义校准，可直接在驱动中实现校准逻辑。

说明

完整的校准流程，参见相应 Android 版本的 Sensor Hub 各类型 Sensor 校准指导手册。

3.8 中断实现

当驱动需要以中断方式进行数据采集上报时，按以下步骤修改驱动：

步骤 1 将 sensor_driver 结构体中的 report_mode 改为 IRQ_MODE。

步骤 2 定义结构体 struct irq_configuration，该结构体用于配置中断信息。

```
struct irq_configuration {
    uint16_t irq_gpio_num;//中断脚的GPIO号
    uint16_t irq_trigger_type;//中断触发类型，在GPIO_INT_TYPE枚举中取值
    uint16_t level_sharking_ms;//为电平触发模式时，需要防抖，则配置该值，不需要则设为0
    BOOLEAN wake_up_system;//中断是否唤醒sensor hub
}
```

步骤 3 实现中断回调函数。该函数为中断上半部，仅处理尽量少的任务。

注意

在中断回调函数中一定要调用 irq_common_handler 函数，并传入该驱动的 sensor_driver_handle 值。irq_common_handler 函数会通知中断处理线程，通过 sensor_driver_handle 找到驱动的 get_data 函数并执行。

步骤 4 在驱动 init 函数中注册中断及中断回调函数。调用 irq_callback_registration() 函数，传入步骤 2 和步骤 3 实现的 irq_configuration 结构体指针和中断回调函数指针。

注意

禁止在中断处理函数中添加打印，因为在中断里面加 log 会存在获取 mutex 互斥锁的问题。RTOS 在执行中断上下文时，任务调度会暂停，不会发生任务切换，直到中断处理完成。由于 mutex 互斥锁的获取存在阻塞操作，如果一个任务获取了互斥锁，其他任务获取时需要等待，而中断实时性较高不能进行阻塞，获取 mutex 就可能导致系统不响应甚至死锁等问题。

----结束

3.9 距感公共逻辑

距感驱动相较于其他驱动更具复杂性，平台对距感的接近/远离计算、动态校准、油污自适应进行了抽象，将其公共逻辑放在 sensor_driver_proximity_common.c 中。如无特殊需求，可在驱动中直接调用公共逻辑。如有特殊需求，可在驱动中自行实现。

3.9.1 距感结构体

3.9.1.1 proximity_cali_params

【功能】

该结构体用于保存距感校准参数设置。

【定义】

```
struct proximity_cali_params {  
    uint16_t ps_threshold_high;  
    uint16_t ps_threshold_low;  
    uint16_t dyna_noise;  
    uint16_t dyna_noise_offset;  
    uint16_t dyna_noise_max;  
    uint16_t noise_high_add;  
    uint16_t noise_low_add;  
    uint16_t ps_threshold_high_def;  
    uint16_t ps_threshold_low_def;  
    uint16_t min_noise;  
    uint16_t max_noise;  
}
```

【成员说明】

成员	说明
ps_threshold_high	保存距感的接近门限值，建议使用多台样机数据中的较大值。当底噪大于该值的时候，就上报接近状态。 开机时获取的校准值及触发动态校准更新后的值都将覆盖此值。 默认值建议设置为 3cm 灰卡采集值。
ps_threshold_low	保存距感的远离门限值，建议使用多台样机数据中的较大值。当底噪小于该值的时候，就上报远离状态。 开机时获取的校准值及触发动态校准计算的值都将覆盖此值。 默认值建议设置为 5cm 灰卡采集值。
dyna_noise	触发动态校准后，保存最新底噪。默认值一般选取遮挡物距离距感 0.1cm 时的距感值。
dyna_noise_offset	底噪浮动值。默认值设为无遮挡时底噪的浮动值。
dyna_noise_max	动态校准时，判断设备是否被遮挡。默认值设置为灰卡 1cm 时的值。
noise_high_add	动态校准时， $\text{noise_high_add} + \text{最新 ground_noise} = \text{新的 ps_threshold_high}$ 。默认值设置为 $(\text{ps_threshold_high 默认值} - \text{ground_noise})$ 的值。
noise_low_add	动态校准时， $\text{noise_low_add} + \text{最新 ground_noise} = \text{新的 ps_threshold_low}$ 。默认值设置为 $(\text{ps_threshold_low 默认值} - \text{ground_noise})$ 的值。
ps_threshold_high_def	取 ps_threshold_high 默认值。当动态校准失效时赋值给 ps_threshold_high。
ps_threshold_low_def	取 ps_threshold_low 默认值。当动态校准失效时赋值给 ps_threshold_low。
min_noise	取正常样机底噪最小值。ground_noise 小于此值就不再进行动态校准。此值可用于工厂异常检出。ground_noise 小于此值时无遮挡校准会失败。
max_noise	默认值应为正常 sensor 底噪最大值，该值将影响产线校准。此值可用于工厂异常检出。ground_noise 大于此值时无遮挡校准会失败。

3.9.1.2 ps_filter

【功能】

该结构体用于在计算远离/接近函数中保存计算的中间值。

【定义】

```
struct ps_filter {
    uint8_t close_flag;
    uint8_t far_away_flag;
    float status_backup;
```

}

【成员说明】

成员	说明
close_flag	保存接近次数，默认值为 0。
far_away_flag	保存远离次数，默认值为 0。
status_backup	保存最新状态，默认值为 5。

注意

需要在 sensor 关闭时将该结构体所有成员置为默认值。

3.9.2 距感公共函数

3.9.2.1 proximity_sensor_process_data

【功能】

通过传入的底噪和结构体中保存的门限值，计算出远离/接近状态。计算出的远离/接近状态保存在 sensor_data 结构体的 data[0] 中。sensor_data 结构体的 data[1] 中保存工厂校准的无遮挡值。该函数通常在 get_data 函数中调用。

【定义】

```
int proximity_sensor_process_data(proximity_cali_params *prox_cali_params, struct ps_filter *filter, struct sensor_data *sensor_data)
```

【参数说明】

参数	说明
prox_cali_params	传入定义的 proximity_cali_params 结构体指针。
filter	传入定义的 ps_filter 结构体指针。
sensor_data	传入 sensor_data 结构体指针，其成员 data[2] 中要保存采集到的底噪值。

【返回值】

- 成功：返回 0，表示 NO_ERROR。
- 失败：返回 1，表示 NO_DATA。

3.9.2.2 proximity_sensor_dynamic_cali

【功能】

该函数在 get_data 函数中调用，在重新使能时根据最新的底噪判断当前状态是否异常，如异常则进行动态校准，计算出新的远离/接近门限值。

【定义】

```
int proximity_sensor_dynamic_cali(proximity_cali_params *prox_cali_params, const float noise)
```

【参数说明】

参数	说明
prox_cali_params	传入定义的 proximity_cali_params 结构体指针。
noise	传入最新的底噪值。

【返回值】

固定返回 0，表示 NO_ERROR。

说明

完整的距感动态校准及校准介绍可参考文档《Android 13 ~ 15 开源 Sensor Hub 距感动态校准设计介绍》。

3.10 光感/距感 raw_data 获取

AP kernel 层的 sensor hub driver 提供有节点 raw_data_als 和 raw_data_ps，可获取实时的光感和距感的原始值。

该功能的实现依赖于 sensor hub 的光感和距感驱动。如需要该功能，需在 sensor hub 光感和距感驱动 get_data 函数中，调用 light_sensor_raw_data_save() 和 proximity_sensor_raw_data_save() 函数，向函数传入获取到的 raw_data 值。传入的 raw_data 值会保存在全局变量中，当读取 raw_data_als 和 raw_data_ps 节点时，会将保存的值发送至 AP。

3.11 sensor 运行在 DDR 上

sensor 运行在 DDR 上可以释放部分 SRAM 空间，可通过在 defconfig 文件配置以下功能宏开启该功能。功能宏配置方法，参见 2.3 配置 sensor hub 功能宏。

```
CONFIG_ORI_FUSION_SENSOR_RUNNING_ON_DDR=y
```


注意

sensor 运行在 DDR 上会导致功耗升高。建议有此需求的 sensor，其使用场景固定为亮屏或 AP 唤醒期间。总体上不推荐使用这种方案，如有此需求，需要自行评估功耗风险。

sensor 运行在 DDR 上分为以下三种情况：

- 原生虚拟 sensor 运行在 DDR 上
平台提供了示例方案，只需开启以上功能宏即可。
- 客制化虚拟 sensor 运行在 DDR 上
除了开启以上功能宏之外，还需在 `bound_and_release_ddr_sleep` 函数中添加 sensor 类型到 case 项。
代码路径：sensor_hub_sprd/public/system/sensor_manager.c
- 物理 sensor 驱动运行在 DDR 上
除了开启以上功能宏之外，还需在 `bound_and_release_ddr_sleep` 函数 case 项中添加该传感器类型（包括“唤醒”和“无唤醒”类型）以及由其合成的虚拟 sensor 类型。
当物理传感器驱动在 DDR 中运行时，驱动文件中的 `struct sensor_driver` 变量应放置在 IRAM 中。以 `light_sip1221lr1n_sensor_driver` 为例，操作步骤如下：

- 修改 `lds` 链接脚本，把 `SENSOR_DRIVER_OBJ` 段放到 IRAM data 区域。

```
/* caculate rw load base addr */
IMG_LD_RW_BASE = LOADADDR(.data);
.data :
{
    . = ALIGN(4);
    __data_start__ = .; /* create a global symbol at data start */
    *(.data) /* .data sections */
    *(.data*) /* .data* sections */
+   *(SENSOR_DRIVER_OBJ) /* SENSOR_DRIVER_OBJ */
    KEEP(*(.jcr*))
    . = ALIGN(4);
    __data_end__ = .; /* define a global symbol at data end */
} > IRAM_RW AT>DDR_CODE
```

- 在 sensor 驱动中通过 `__attribute__` 把 `light_sip1221lr1n_sensor_driver` 定向到 `SENSOR_DRIVER_OBJ` 段。

```
static struct sensor_driver light_sip1221lr1n_sensor_driver
__attribute__((__section__("SENSOR_DRIVER_OBJ"))) = {
    .sensor_info = &light_sip1221lr1n_info,
    ...
}
```

-----结束

4 sensor hub 客制化指导

4.1 sensor bring up

4.1.1 新增 sensor hub 驱动

当新的 sensor hub 驱动代码准备好后，可按如下步骤将驱动加入 sensor hub 工程编译：

步骤 1 若新增的 sensor 类型之前未支持，需要包含该 sensor 的编译文件和配置：

- 在 CMakeList.txt 中参照已有的 sensor 类型配置增加该 sensor 类型的.cmake。
文件路径：
 - sp sys: bsp/sensorhub/public/sensor_hub_sprd/public/CMakeList.txt
 - ch sys: bsp/contexthub/modules/sensorhub/sensor_hub_sprd/public/CMakeList.txt
- 在 Kconfig 中参照已有的 sensor 类型配置增加该 sensor 类型的 Kconfig。
文件路径：
 - sp sys: bsp/sensorhub/public/sensor_hub_sprd/public/Kconfig
 - ch sys: bsp/contexthub/modules/sensorhub/sensor_hub_sprd/public/Kconfig

步骤 2 增加该型号 sensor 的功能宏。

在该类型的驱动文件目录找到 Kconfig 文件，参照已有的宏增加新型号的功能宏。

步骤 3 增加该型号 sensor 的功能宏的默认配置。

在各项目的 defconfig 文件中，将该型号 sensor 的功能宏默认配置为关闭。

说明

新增控制宏默认值需配为关闭，避免其他项目将新增的驱动编译进工程，导致内存浪费。需要编译该驱动的工程，通过配置工程 defconfig 文件，将该功能宏打开即可。详细信息，参见 [2.3 配置 sensor hub 功能宏](#)。

步骤 4 将驱动加入 sensor hub 工程编译。

在该类型的驱动文件目录找到.cmake 文件，使用 list(APPEND SRCS xxx)追加驱动文件到 SRCS，并用新增的功能宏控制该修改。使用 cp_library_include_directories()声明需要用到的头文件路径。使用 cp_append_export_library()声明需要用到的库文件。

说明

若在支持 overlay 的 board 中新增 sensor，需在 lds 文件中增加该型号 sensor 的配置。详细信息，参见 [3.6 overlay 实现](#)。

----结束

4.1.2 配置 AP 端 sensor 类型宏

AP 代码 device/sprd/(项目名)/(board 名)/module/sensor/md.mk 文件中配置支持的 sensor 类型，配置如下：

```
SENSOR_HUB_ACCELEROMETER := true
SENSOR_HUB_GYROSCOPE := true
SENSOR_HUB_LIGHT := true
SENSOR_HUB_MAGNETIC := true
SENSOR_HUB_PROXIMITY := null
SENSOR_HUB_PRESSURE := true
```

- true：支持该类型 sensor
- null：不支持该类型 sensor

说明

此处的宏配置将影响传感器支持列表 sensor_list。

4.2 Kernel 5.15 版本添加 AP 与 sensor hub 交互命令

AP 与 sensor hub 之间通过共享内存（SIPC）的方式进行交互，通过一端往共享内存中写入消息，另一端去共享内存中取走消息完成通信。

按如下步骤添加 AP 与 sensor hub 交互命令：

步骤 1 在 AP 和 sensor hub 中增加新的消息类型。

- sensor hub：在以下文件的 SHUB_OPCODE_ID 中增加新的枚举
文件路径：
 - sp 平台：
bsp/sensorhub/public/sensor_hub_sprd/public/source/common/protocol/inc/sensor_hub_ap_core.h
 - ch 平台（UMS9632 和 UMS9360 除外）：
bsp/contexthub/modules/sensorhub/sensor_hub_sprd/public/source/common/protocol/inc/sensor_hub_ap_core.h
- AP：在以下文件的 shub_shubtype_id 中增加相同的枚举
文件路径：bsp/(kernel 版本)/(kernel 版本)/drivers/iio/sprd_hub/shub_protocol.h

步骤 2 调用 AP 和 sensor hub 中的发送消息函数发送新增类型消息。

步骤 3 在 AP 和 sensor hub 接收消息函数处增加对新增消息的处理。

-----结束

说明

目前 UMS9632 和 UMS9360 搭配 Kernel 6.6，其余平台搭配 Kernel 5.15。请注意 Kernel 版本与平台的搭配关系。

4.2.1 SIPC 消息格式

SIPC 消息格式通过以下结构体定义。

【定义】

```
struct{
    u8 type;
    u8 subtype;
    u16 length;
    u8 buffer[];
}
```

【成员说明】

成员	说明
type	发送消息的 sensor 类型值。
subtype	消息类型。
length	消息长度。
buffer[]	消息内容。

4.2.2 AP 发送 sensor hub 接收

4.2.2.1 AP 发送消息函数

4.2.2.1.1 shub_send_command

【功能】

发送消息至 sensor hub。

【定义】

```
static int shub_send_command(struct shub_data *sensor, int sensor_ID, enum shub_subtype_id opcode, const char *data, int len)
```

【参数说明】

参数	说明
sensor	sensor hub driver shub_data 结构体指针。
sensor_ID	sensor 类型，赋给 SIPC 消息的 type。
opcode	shub_subtype_id 枚举，赋给 SIPC 消息的 subtype。

参数	说明
data	发送的消息指针，赋给 SIPC 消息的 buffer。
len	发送消息的长度，赋给 SIPC 消息的 length。

【返回值】

- 成功：返回发送消息个数。
- 失败：返回无设备错误码。

4.2.2.1.2 shub_sipc_read

【功能】

发送消息至 sensor hub 并等待 sensor hub 回应。

【定义】

```
static int shub_sipc_read(struct shub_data *sensor, u8 reg_addr, u8 *data, u32 len)
```

【参数说明】

参数	说明
sensor	sensor hub driver shub_data 结构体指针。
reg_addr	shub_shubtype_id 枚举，赋给 SIPC 消息的 subtype。
data	发送的消息指针，赋给 SIPC 消息的 buffer。
len	发送消息的长度，赋给 SIPC 消息的 length。

【返回值】

- 成功：返回 0。
- 失败：返回超时错误。sensor hub 在 100ms 内未回应该类型消息则返回失败。

4.2.2.2 sensor hub 处理接收的消息

gHandleCmdTable[] 结构体数组中增加新的命令及其处理函数。

- 不耗时的或优先级较高的操作函数可直接注册到 gHandleCmdTable[] 结构体中。
- 较耗时的操作，将其处理函数注册为 SensorhubCommandParse()，并在 sensor_ap_cmd_handle() 函数内增加其操作逻辑。
- sensor_ap_cmd_handle() 在优先级较低的一条线程中被执行。

4.2.3 sensor hub 发送 AP 接收

4.2.3.1 sensor hub 发送消息函数

4.2.3.1.1 SIPCADPTR_SendWakeUpData

【功能】

发送消息至 AP，会唤醒 AP。

【定义】

```
int SIPCADPTR_SendWakeUpData(void *psrc, uint16_t length, uint8_t type, uint8_t sub_type)
```

【参数说明】

参数	说明
psrc	发送的消息指针，赋给 SIPC 消息的 buffer。
length	发送消息的长度，赋给 SIPC 消息的 length。
type	sensor 类型，赋给 SIPC 消息的 type。
sub_type	SHUB_OPCODE_ID 中的枚举，赋给 SIPC 消息的 subtype。

【返回值】

返回 0。

4.2.3.1.2 SIPCADPTR_SendUnWakeUpData

【功能】

发送消息至 AP，不会唤醒 AP。

【定义】

```
int SIPCADPTR_SendUnWakeUpData(void *psrc, uint16_t length, uint8_t type, uint8_t sub_type)
```

【参数说明】

参数	说明
psrc	发送的消息指针，赋给 SIPC 消息的 buffer。
length	发送消息的长度，赋给 SIPC 消息的 length。
type	sensor 类型，赋给 SIPC 消息的 type。
sub_type	SHUB_OPCODE_ID 中的枚举，赋给 SIPC 消息的 subtype。

【返回值】

返回 0。

4.2.3.2 AP 处理接收的消息

在 static void shub_get_data(struct cmd_data *packet)函数中对消息类型进行判断后执行对应操作。

4.3 Kernel 6.6 版本添加 AP 与 sensor hub 交互命令

AP 与 sensor hub 之间通过共享内存（SIPC）的方式进行交互，通过一端往共享内存中写入消息，另一端去共享内存中取走消息完成通信。

按如下步骤添加 AP 与 sensor hub 交互命令：

步骤 1 在 AP 和 sensor hub 中增加新的消息类型。

- sensor hub：在以下文件的 SHUB_OPCODE_ID 中增加新的枚举
文件路径：
bsp/contexthub/modules/sensorhub/sensor_hub_sprd/public/source/common/protocol/inc/sensor_hub_ap_core.h
- AP：在以下文件的 shub_shubtype_id 中增加相同的枚举
文件路径：bsp/(kernel 版本)/unicoc/drivers/unisoc_platform/sprd_hub/shub_core.h

步骤 2 调用 AP 和 sensor hub 中的发送消息函数发送新增类型消息。

步骤 3 在 AP 和 sensor hub 接收消息函数处增加对新增消息的处理。

----结束

说明

目前 UMS9632 和 UMS9360 搭配 Kernel 6.6，其余平台搭配 Kernel 5.15。请注意 Kernel 版本与平台的搭配关系。

4.3.1 SIPC 消息格式

SIPC 消息格式通过以下结构体定义。

【定义】

```
struct{  
    enum shub_system_id sys_id;  
    u8 cfg;  
    u16 length;  
    u8 buffer[];  
}
```


【成员说明】

成员	说明
sys_id	接收消息的子系统的 ID。
cfg	子系统的私有配置，sensor hub 子系统信息如下： • 发送消息的 sensor 类型值 • 消息类型
length	消息长度。
buffer[]	消息内容。

4.3.2 AP 发送 sensor hub 接收

4.3.2.1 AP 发送消息函数

4.3.2.1.1 shub_sipc_send_cmd

【功能】

发送消息至 sensor hub。

【定义】

```
int shub_sipc_send_cmd(enum shub_system_id sys_id, u16 cfg, const char *data, u32 len)
```

【参数说明】

参数	说明
sys_id	子系统 ID，shub_system_id 枚举类型，赋给 SIPC 消息的 sys_id。
cfg	子系统的私有配置，赋给 SIPC 消息的 cfg。
data	发送的消息指针，赋给 SIPC 消息的 buffer。
len	发送消息的长度，赋给 SIPC 消息的 length。

【返回值】

- 成功：返回发送消息个数。
- 失败：返回无设备错误码。

4.3.2.1.2 shub_sipc_cmd_read

【功能】

发送消息至 sensor hub 并等待 sensor hub 回应。

【定义】

```
int shub_sipc_cmd_read(enum shub_system_id sys_id, u16 cfg, u8 *data, u32 len)
```

【参数说明】

参数	说明
sys_id	子系统 ID, shub_system_id 枚举类型, 赋给 SIPC 消息的 sys_id。
cfg	子系统的私有配置, 赋给 SIPC 消息的 cfg。
data	发送的消息指针, 赋给 SIPC 消息的 buffer。
len	发送消息的长度, 赋给 SIPC 消息的 length。

【返回值】

- 成功: 返回 0。
- 失败: 返回超时错误。sensor hub 在 100ms 内未回应该类型消息则返回失败。

4.3.2.2 sensor hub 处理接收的消息

gHandleCmdTable[] 结构体数组中增加新的命令及其处理函数。

- 不耗时的或优先级较高的操作函数可直接注册到 gHandleCmdTable[] 结构体中。
- 较耗时的操作, 将其处理函数注册为 SensorhubCommandParse(), 并在 sensor_ap_cmd_handle() 函数内增加其操作逻辑。
- sensor_ap_cmd_handle() 在优先级较低的一条线程中被执行。

4.3.3 sensor hub 发送 AP 接收

4.3.3.1 sensor hub 发送消息函数

4.3.3.1.1 SIPCADPTR_SendWakeUpData

【功能】

发送消息至 AP, 会唤醒 AP。

【定义】

```
int SIPCADPTR_SendWakeUpData(void *psrc, uint16_t length, enum shub_system_id sys_id, uint16_t config)
```

【参数说明】

参数	说明
psrc	发送的消息指针，赋给 SIPC 消息的 buffer。
length	发送消息的长度，赋给 SIPC 消息的 length。
sys_id	子系统 ID，shub_system_id 枚举类型，赋给 SIPC 消息的 sys_id。
config	子系统的私有配置，赋给 SIPC 消息的 cfg。

【返回值】

返回 0。

4.3.3.1.2 SIPCADPTR_SendUnWakeUpData

【功能】

发送消息至 AP，不会唤醒 AP。

【定义】

```
int SIPCADPTR_SendUnWakeUpData(void *psrc, uint16_t length, enum shub_system_id sys_id, uint16_t config)
```

【参数说明】 【参数说明】

参数	说明
psrc	发送的消息指针，赋给 SIPC 消息的 buffer。
length	发送消息的长度，赋给 SIPC 消息的 length。
sys_id	shub_system_id 枚举类型，对应子系统的 ID。
config	子系统的私有配置，sensor hub 子系统上包含的信息有： - sensor 类型，赋给 SIPC 消息的 type。 - sub_type 赋给 SIPC 消息的 subtype

【返回值】

返回 0。

4.3.3.2 AP 处理接收的消息

在 static void shub_get_data(struct cmd_data *packet)函数中对消息类型进行判断后执行对应操作。

4.4 移除 sensor

当项目不需要支持 sensor 功能时，可按如下步骤将 sensor 相关内容移除：

步骤 1 移除 sensor hub driver。

在 AP 代码中将对应项目的 CONFIG_SPRD_SENSOR_HUB 宏关闭。具体配置步骤如下：

- 进入 bsp 目录。
- 执行命令 `source build/envsetup.sh`。
- 执行命令 `lunch`，选择工程。
- 执行命令 `kuconfig`，将弹出配置窗口。
- 在弹出的配置界面中，依次选择 Device Drivers > Industrial I/O support > Spreadtrum Sensor Hub，去掉勾选 Spreadtrum Sensor Hub。
- 根据页面提示保存并退出。

步骤 2 移除 sensor hub 设备。

在 AP 对应项目的 DTS 中移除 sprd-sensorhub 设备节点，即在 `bsp/(kernel 版本)/(kernel 版本)/arch/arm64/boot/dts/sprd/(项目名)/` 目录中删掉文件 `sensorhub.dtsi`，并在 `(项目名).dtsi` 中去掉 `#include "sensorhub.dtsi"`。

步骤 3 移除 sensor hub HAL 层代码。

在 AP 代码中将对应项目的 USE_SPRD_SENSOR_HUB 宏关闭，即在 `device/sprd/vnd_mpool/module/vendor/sensor/main.mk` 中将 `USE_SPRD_SENSOR_HUB` 设置为 `false`，并将 `android.hardware.sensors-service.multihal` 和 `android.hardware.sensors@2.X-sprd_subhal` 从 `PRODUCT_PACKAGES` 中去除。

----结束

5 Debug 方法

5.1 sensor hub log 获取

5.1.1 adb 获取实时 log

获取实时 log 的 adb 命令如下：

- sensor hub HAL 实时 log: `logcat | grep SensorHub`
- sensor hub kernel 实时 log: `cat proc/kmsg | grep sensorhub` 或 `dmesg -w | grep sensorhub`
- sensor hub 系统实时 log:
 - sp sys: `cat dev/slog_pm`
 - ch sys: `cat dev/slog_ch`

5.1.2 获取历史 log

- 如果开机前未插 SD 卡：
 - sp sys: sensor hub log 保存在 `data/ylog/pm_sensorhub` 中。
 - ch sys: sensor hub log 保存在 `data/ylog/sensorhub` 中。
- 如果开机前插有 SD 卡: sensor hub log 默认保存在 SD 卡中的 `ylog` 目录。

说明

`data/ylog` 目录中的 `AP` 目录下还保存有 `Android log` 和 `kernel log`。

5.2 常用系统文件节点调试 sensor hub 命令

常用的系统文件节点调试命令如表 5-1 所示：

表5-1 常用的系统文件节点调试命令

adb 调试命令	功能说明
• sp sys: – <code>echo "log on" > dev/sctl_pm</code>: 打开 – <code>echo "log off" > dev/sctl_pm</code>: 关闭 • ch sys: – <code>echo "log on" > dev/sctl_ch</code>: 打开 – <code>echo "log off" > dev/sctl_ch</code>: 关闭	手动打开或关闭 sensor hub log。

adb 调试命令	功能说明
cat sys/class/sprd_sensorhub/sensor_hub/version	获取 sensor hub 版本信息。 若无法获取，则说明 sensor hub 核工作异常。 版本显示时间为 sensor hub 工程编译时间
cat sys/class/sprd_sensorhub/sensor_hub/sensor_info	获取 sensor hub 上挂载成功的 sensor 信息。 其信息来源于 sensor hub 驱动的 sensor_info_t 结构体。
cat sys/class/sprd_sensorhub/sensor_hub/raw_data_als(ps)	获取光感（距感）实时未校准数据。 注意 需要在 sensor 打开状态时执行该命令，否则获取的数据为上一次的值，不会更新。 若之前从未打开过，没有值可读，将导致 cat 显示超时。
echo "4 2" > sys/class/sprd_sensorhub/sensor_hub/logctl	设置 sensor hub HAL 层 log 打印等级为 4。 可用 logcat 获取到实时上报的 sensor 事件。
echo "handle enable" > sys/class/sprd_sensorhub/sensor_hub/enable	手动开关 sensor 的命令格式。
echo "handle 0 period_us timeout"> sys/class/sprd_sensorhub/sensor_hub/batch	手动设置 sensor 采样率的命令格式。
- sp sys: echo "assert on" > dev/sctl_pm - ch sys: echo "assert on" > dev/sctl_ch	主动 assert sensor hub。
echo "op intf addr reg value mask" > sys/class/sprd_sensorhub/sensor_hub/cm4_operate cat sys/class/sprd_sensorhub/sensor_hub/cm4_operate	该节点可进行 sensor 寄存器读写、操作 sensor hub GPIO、手动设置延时。 可先执行 cat cm4_operate 函数获取操作提示。 执行写操作后再读该节点，可获得执行状态和操作返回结果。
1. 配置 SPI 总线： echo "bus_num set_op freq cs mode bit_per_word" > sys/class/sprd_sensorhub/sensor_hub/cm4_operate 查看 SPI 总线设置状态： cat sys/class/sprd_sensorhub/sensor_hub/cm4_spi_set 2. 读/写 SPI sensor 寄存器： echo "op spi_sync reg_addr value1 value2 len" > sys/class/sprd_sensorhub/sensor_hub/cm4_operate 查看操作返回结果： cat sys/class/sprd_sensorhub/sensor_hub/cm4_spi_sync	在线读写 SPI，需要先配置要使用的 SPI 总线，配置成功后，再进行 SPI 读写。 操作 SPI 节点前，可先分别执行 cat cm4_spi_set 和 cat cm4_spi_sync 获取各步骤操作提示和参数说明。

6

常见问题及处理方法

6.1 sensor hub 工程编译提示内存段溢出错误

6.1.1 问题现象

如下图编译报错，提示 IRAM_CODE 段内存超出。

```
e-eabi/bin/ld: region `IRAM_CODE' overflowed by 5000 bytes  
collect2: error: ld returned 1 exit status
```

6.1.2 问题处理

可在编译时将 log 导出（如：使用 `make sensorhub 2>&1 | tee build.log` 命令），在 log 中找到 memory region 列表，查看各段实际使用情况：

- IRAM 整体不超出

如下图可见 IRAM_RW 剩余内存远超 IRAM_CODE 的内存，直接在以下文件中增加 CODE 段大小，扩大 IRAM_CODE 段的内存即可解决问题。

文件路径：

- sp sys: bsp/sensorhub/public/ms_customize/source/product/config/(项目名)/mem_cfg.h
- ch sys: bsp/contexthub/device/CH/(项目名)/(board 名)/mem_cfg.h

Memory region	Used Size	Region Size	%age Used
IRAM:	0 GB	312 KB	0.00%
IRAM_CODE:	168840 B	160 KB	103.05%
IRAM_HEAP:	28 KB	28 KB	100.00%
IRAM_RW:	103056 B	124 KB	81.16%
DDR:	0 GB	2 MB	0.00%
DDR_CODE:	637972 B	1 MB	61.80%
DDR_HEAP:	0 GB	256 KB	0.00%
DDR_RW:	366876 B	768 KB	46.65%

说明

如果 IRAM 整体不超出，但 IRAM_RW 段超出时，则需要减少 CODE 段大小。

- IRAM 整体超出

如下图 IRAM_CODE 和 IRAM_RW 的 Used Size 之和大于 Region Size 之和，即 IRAM 内存整体超出了。这种情况通过调整各段内存大小已经不能解决问题，只能通过移除部分 sensor 驱动、关闭部分功能或者优化代码来减少内存占用。

Memory region	Used Size	Region Size	%age Used
IRAM:	0 GB	312 KB	0.00%
IRAM_CODE:	191528 B	160 KB	116.90%
IRAM_HEAP:	28 KB	28 KB	100.00%
IRAM_RW:	105496 B	124 KB	83.08%
DDR:	0 GB	2 MB	0.00%
DDR_CODE:	672992 B	1 MB	64.18%
DDR_HEAP:	0 GB	256 KB	0.00%
DDR_RW:	366876 B	768 KB	46.65%

7

参考文档

1. 对应 Android 版本的 Sensor Hub 各类型 Sensor 校准指导手册
2. 《Android 13~16 开源 Sensor Hub 距感动态校准设计介绍》